«Университет Иннополис»
(АНО ВО «Университет Иннополис»)

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 – «Информатика и вычислительная техника»

GRADUATION THESIS
(BACHELOR'S GRADUATE THESIS)
Field of Study
09.03.01 – «Computer Science»

Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»
Area of Specialization / Academic Program Title:
«Computer Science»

Тема / Topic	**Система уточняющих типов для языка Dart / Refinement type system for Dart**

Работу выполнил / Thesis is executed by	**Чистяков Климентий Павлович /** **Chistyakov Klimentii Pavlovich**	подпись / signature
Руководитель выпускной квалификационной работы / Graduation Thesis Supervisor	**Николай Кудасов Дмитриевич /** **Nikolai Kudasov Dmitrievich**	подпись / signature
Консультанты /		



Иннополис, Innopolis, 2026

Contents

1 Introduction **7**

1.1 Background & Relevance 7

1.2 Problem Statement 8

1.3 Objectives 8

1.4 Limitations & Scope 8

2 Literature Review **10**

2.1 Refinement Types 10

2.2 Liquid Types and Refinement Type Inference 11

2.3 Refinement Types in Other Languages 11

2.4 SMT Solvers and Z3 12

3 Methodology **14**

4 Implementation **16**

5 Evaluation and Discussion **17**

6 Conclusion **18**

Bibliography cited **19**

A Extra Stuff **22**

B Even More Extra Stuff **23**

List of Tables

3.1 This is the title I want to appear in the List of Tables 14

List of Figures

Abstract

My abstract starts from here.

Chapter 1

Introduction

1.1 Background & Relevance

Index out of bounds errors are a well known reason for critical bugs appearing in modern software, mainly occuring in places where the checks for proper indexing are either insufficient or lacking entirely. The most notable example of these errors is access to an element of array that doesn't exist at the index provided.

Since a lot of programming languages rely on the array as a commonly used data structure, it is easy to encounter such an error in any part of the developed software. The examples include but are not limited to color conversions [need citation 89584 changelog], pixel values [need citation 8] or handling of empty lists.

Unfortunately, due to the nature of how array accesses happen, these errors are quite difficult to notice until they reach runtime or sometimes even production environments completely unnoticed.

1.2 Problem Statement

Dart programming language is prone to index out of bounds errors. The current type system, though incressing robustness [citation dart language specs], does not currently support collection-length constraints. This makes developers rely on assumptions and runtime testing.
This paper attempts to resolve this issue by introducing a refinement type system to statically confirm safety of array access operations while adding little annotation burden to the developers.

1.3 Objectives

We aim to achieve the following goals:

- An implementation of an analyzer for a custom predicate language
- An implementation of a refinement type checker and type inference algorithm with usage of Z3 constraints solver
- Integration of said checker into Dart analysis workflow [1]
- Evaluation of the effectivness of the implementation on a package corpus from pub.dev

1.4 Limitations & Scope

The implementation will cover the following features of Dart and will allow users to write the following refinements:

- Refinement type definitions that will allow users to extend integers, booleans and lists with refinements
- Definition of variables with refinement types
- Definition of recursive/non-recursive functions
- Usage of $length(xs)$ function in refinements for List type and its refinement extensions

Chapter 2

Literature Review

2.1 Refinement Types

Refinement types allow the developer to extend a language's type system with predicates [2]. For example, the user can constraint the regular integer type with the following refinement to get the type that represents only natural numbers.

```
1  @Refinement(identifier: 'x', predicate: 'x >= 0')
2  typedef nat = int;
```

With such refinements the developer can easily avoid index out of bounds errors by, for example, defining the following refinement:

```
1  @Refinement(parameters: ['list', 'index'], predicate: 'index
       <= length(list)')
2  T get(List<T> list, int index) => list[index];
```

The type checker will ensure that the index provided to the function is smaller than the length of the list, and if not, will report this operation as unsafe.

2.2 Liquid Types and Refinement Type Inference

Annotating each variable, each type with with refinement information can get overwhelming and consume more time than the annotations will reduce in error finding. To address this issue we can implement refinement type inference using *Logically Qualified Data Types* also known as *Liquid Types* system explored by *Rondon et al.*[3] which will provide the developer with strength of refinement types without additional unnecessary effort. The inference process involves the following steps:

1. Hindley-Milner type inference [4]
2. Constraint generation
3. Constraint solving using an SMT solver [5]

Assume a function *safeHead*, where we want to check safety of *xs[0]* operation:

```
1    int safeHead(List<int> xs) => xs.length > 0 ? xs[0] : 0;
```

Without type inference we would need to manually specify length property of *xs* as a refinement, however liquid types can infer the constraint from the program itself.

2.3 Refinement Types in Other Languages

Similar implementations exist for other programming languages. The most similar one to my implementation is a refinement type system for TypeScript [6] by *Verkis et al*. Their work demonstrates an integration of a refinement type system into an imperative object-oriented language while retaining support for

higher-order functions.

One of the main problems the implementation faces is treatment of mutable variables. Since the value of a variable can change during program execution, refinement predicates can become invalid after reassignment. To address the issue the authors translate input programs into Static Single Assignment form, where each variable is assigned only once. This allows their checker to reason about individual variable versions and retain the information about conditions under which they were assigned.

The implementation also uses the Liquid Types [3] system which, on par with TypeScript being similar to Dart on many accounts [7], makes this implementation a good reference point for my project.
Other notable examples of such system implementations are [8, Liquid Haskell], [9, Refinement Types For Ruby] , and [10, Flux: Liquid Types for Rust]

2.4 SMT Solvers and Z3

Satisfiability Modulo Theories (SMT) is a problem that asks whether a first-order logic formula is satisfiable with respect to a background theory [11]. It extends Boolean Satisfiability (SAT) problem by adding reasoning such as integers, arrays, bit-verctos and uninterpreted functions. Since SMT solvers are able of reasoning about relationships between values and operations within theories, they become particularly useful in software verification, where program properties involve arithmetic constraints, equality connections and data structures. de Moura and Bjørner present Z3 [5], an SMT solver developed at Microsoft

Research and designed specifically for applications in software verification. In context of refinement type checking, an SMT solver can serve as the reasoning engine, responsible for verifying refinement predicates and generating constraints, making Z3 suitable for proposed refinement type checker for Dart. Using Z3 allows to delegate logical reasoning to Z3 and focus on the implementation on extracting extracting properties and interpreting verification results.

Chapter 3

Methodology

Referencing other chapters 2, 3, 4, 5 and 6

TABLE 3.1
Simulation Parameters

A	B
Parameter	**Value**
Number of vehicles	$\mid \mathcal{V} \mid$
Number of RSUs	$\mid \mathcal{U} \mid$
RSU coverage radius	150 m
V2V communication radius	30 m
Smart vehicle antenna height	1.5 m
RSU antenna height	25 m
Smart vehicle maximum speed	v_{max} m/s
Smart vehicle minimum speed	v_{min} m/s
Common smart vehicle cache capacities	$[50, 100, 150, 200, 250]$ mb
Common RSU cache capacities	$[5000, 1000, 1500, 2000, 2500]$ mb

A	B
Common backhaul rates	$[75, 100, 150]$ mb/s

...

Chapter 4

Implementation

...

Chapter 5

Evaluation and Discussion

...

Chapter 6

Conclusion

...

Bibliography cited

[1] *Analyzer plugins*, en. Accessed: Sep. 28, 2026. [Online]. Available: https://dart.dev/tools/analyzer-plugins

[2] R. Jhala and N. Vazou, *Refinement Types: A Tutorial*, en, arXiv:2010.07763 [cs.PL], Oct. 2020. DOI: 10.48550/arXiv.2010.07763 Accessed: Oct. 4, 2026. [Online]. Available: http://arxiv.org/abs/2010.07763

[3] P. M. Rondon, M. Kawaguci, and R. Jhala, "Liquid types," in *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '08, New York, NY, USA: Association for Computing Machinery, Jun. 2008, pp. 159–169, ISBN: 978-1-59593-860-2. DOI: 10.1145/1375581.1375602 Accessed: Oct. 4, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/1375581.1375602

[4] L. Damas and R. Milner, "Principal Type-Schemes for Functional Programs," en,

[5] L. de Moura and N. Bjørner, "Z3: An Efficient SMT Solver," en, in *Tools and Algorithms for the Construction and Analysis of Systems*, C. R. Ramakrishnan and J. Rehof, Eds., Berlin, Heidelberg: Springer, 2008, pp. 337–340, ISBN: 978-3-540-78800-3. DOI: 10.1007/978-3-540-78800-3_24

[6] P. Vekris, B. Cosman, and R. Jhala, "Refinement types for TypeScript," in *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '16, New York, NY, USA: Association for Computing Machinery, Jun. 2016, pp. 310–325, ISBN: 978-1-4503-4261-2. DOI: 10.1145/2908080.2908110 Accessed: Oct. 4, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/2908080.2908110

[7] *Dart-lang/language*, original-date: 2018-06-13T22:49:18Z, Sep. 2026. Accessed: Sep. 28, 2026. [Online]. Available: https://github.com/dart-lang/language

[8] N. Vazou, E. L. Seidel, and R. Jhala, "LiquidHaskell: Experience with refinement types in the real world," in *Proceedings of the 2014 ACM SIGPLAN symposium on Haskell*, ser. Haskell '14, New York, NY, USA: Association for Computing Machinery, Sep. 2014, pp. 39–51, ISBN: 978-1-4503-3041-1. DOI: 10.1145/2633357.2633366 Accessed: Oct. 4, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/2633357.2633366

[9] M. Kazerounian, N. Vazou, A. Bourgerie, J. S. Foster, and E. Torlak, "Refinement Types for Ruby," en, in *Verification, Model Checking, and Abstract Interpretation*, I. Dillig and J. Palsberg, Eds., Cham: Springer International Publishing, 2018, pp. 269–290, ISBN: 978-3-319-73721-8. DOI: 10.1007/978-3-319-73721-8_13

[10] N. Lehmann, A. T. Geller, N. Vazou, and R. Jhala, "Flux: Liquid Types for Rust," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, 169:1533–169:1557, Jun. 2023. DOI: 10.1145/3591283 Accessed: Oct. 4, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3591283

[11] C. Barrett, R. Sebastiani, S. A. Seshia, and C. Tinelli, "Chapter 33. Satisfiability Modulo Theories," en, in *Handbook of Satisfiability*, ser. Frontiers in Artificial Intelligence and Applications, A. Biere, M. Heule, H. Van Maaren, and T. Walsh, Eds., IOS Press, Feb. 2021, ISBN: 978-1-64368-160-3 978-1-64368-161-0. DOI: 10.3233/FAIA201017 Accessed: Oct. 30, 2026. [Online]. Available: http://ebooks.iospress.nl/doi/10.3233/FAIA201017

[12] J. Dunfield and N. Krishnaswami, "Bidirectional Typing," *ACM Computing Surveys (CSUR)*, vol. 54, no. 5, 98:1–98:38, May 2021, ISSN: 0360-0300. DOI: 10.1145/3450952 Accessed: Sep. 28, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3450952

[13] *Flux: Liquid Types for Rust | Proceedings of the ACM on Programming Languages*. Accessed: Oct. 4, 2026. [Online]. Available: https://dl.acm.org/doi/abs/10.1145/3591283

[14] C. Gamboa, A. Reese, A. Fonseca, and J. Aldrich, "Usability Barriers for Liquid Types," *Proc. ACM Program. Lang.*, vol. 9, no. PLDI, 224:1911–224:1936, Jun. 2025. DOI: 10.1145/3729327 Accessed: Oct. 4, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3729327

[15] C. Gamboa, A. Reese, A. Fonseca, and J. Aldrich, "Usability Barriers for Liquid Types," *Proc. ACM Program. Lang.*, vol. 9, no. PLDI, 224:1911–224:1936, Jun. 2025. DOI: 10.1145/3729327 Accessed: Oct. 4, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3729327

[16] *Documentation - Nanopass Framework*. Accessed: Oct. 12, 2026. [Online]. Available: https://nanopass.org/documentation.html

Appendix A

Extra Stuff

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Appendix B

Even More Extra Stuff

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.